변재혁 · 백엔드 개발자 8년차

부하 테스트로 병목을 찾아 **구조를 바꿔** 해결해 온 백엔드 개발자
지금은 그룹웨어 **LLM** 에이전트 서비스를 운영하고 있습니다

Java · Spring | Python · FastAPI | LangGraph | PostgreSQL | Airflow | RabbitMQ | Kubernetes

DB 작업 7 → 0

요청당 체크포인트 DB 작업 제거
부하 시 PostgreSQL 장애 원인 해소

AI 에이전트 · 2026

약 200TB

클러스터 9개 첨부 이관 파이프라인
미이관 0.2%는 사용하지 않는 파일

데이터 마이그레이션 · 2025–26

C → Java

4.0 메일 스토어 신규 구축
클러스터당 2대, 증설 가능한 구조

메일 스토어 · 2024–25

8,000명 / 10분

1만 명 고객사 출근 피크 부하 테스트
근태 표시 ON 장애 → 정상

성능 안정화 · 2023–24

커리어 타임라인



AI 에이전트 · 개요

2026. 7 ~ 현재

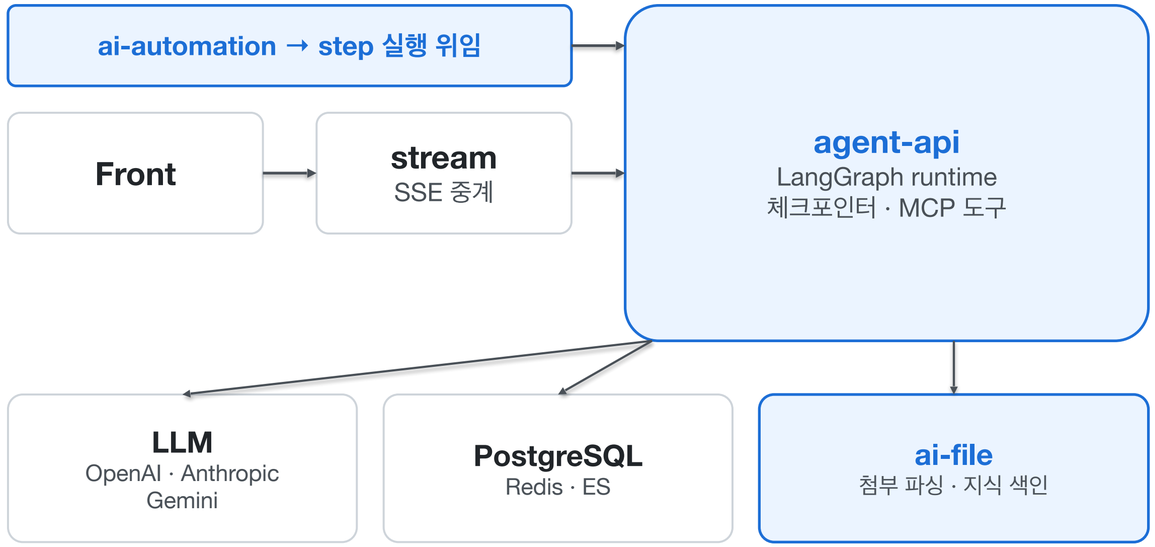
그룹웨어 AI 에이전트 운영

배경

- 메일·결재·캘린더·메신저를 도구로 호출하는 LangGraph ReAct 에이전트
- 외주사가 만든 POC를 **오픈 시점부터 인수해 운영과 성능 개선 담당**

역할

- 팀 5명 · agent 개발은 본인 + 외주 2명, **운영은 본인 담당**
- 운영 장애 대응, 부하테스트로 계층별 병목을 찾아 DB·메모리 문제 개선
- 메신저·전자결재 MCP 도구를 에이전트에 연동 (MCP 서버는 타 팀원)
- 신규 업무 자동화 서비스(ai-automation) 설계·개발



파란 박스: 직접 개발·운영한 영역

주요 기여

체크포인터 재설계

부하 시 DB를 무너뜨리던 구조 해결
(다음 장)

MCP 도구 연동

메신저·전자결재 도구를 에이전트에 연결
결재 카드·간편결재 재시도 흐름

부하테스트 병목 규명

agent·stream·DB 계층별 병목 판정
ai-file 메모리 989MB → 280MB

ai-automation 재설계·구현

DB 큐 → MQ 중심 실행 구조
(진행 중)

Python · FastAPI · LangGraph · OpenAI / Anthropic / Gemini · PostgreSQL · Redis · RabbitMQ · Elasticsearch · k8s(KEDA)

AI 에이전트 · 문제 해결

2026. 7 ~ 8

부하 시 DB를 무너뜨리던 체크포인트 구조 재설계

① 문제

- 부하테스트 시작 약 6분 만에 PostgreSQL이 모든 파드 커넥션 절단 (6,000VU, 4,000VU에서도 재발)
- checkpoint_blobs 27GB, 디스크가 가득 차 PG 중단
- DBA가 현재 구조로는 운영 불가로 반려

② 원인

- 요청마다 체크포인트를 새로 쓰고, 끝나면 바로 삭제
- 단계마다 대화 전체를 다시 저장 → 턴당 102MiB (라운드 수의 제곱으로 증가)
- 첨부 base64까지 체크포인트에 실려 매 요청 DB에 적재

③ 해결

- 1차: 종료 때 1회만 저장 (102 → 18.5MiB), 쓰고 지우는 구조는 남아 DB 재중단
- 2차: 일반 run은 프로세스 메모리에만 보관 → 첨부 적재도 함께 사라짐
- 승인 대기로 멈춘 run만 PG에 저장 → 다른 파드에서 재개
- 파드 종료 시 graceful shutdown(최대 120초)으로 진행 중 run을 마저 처리

④ 결과

요청당 DB 작업
7건 → 0건

- 일반 요청의 체크포인트 DB 작업 제거
- 개선 후 3,000VU에서 DB 오류 0, 다음 병목은 stream CPU (코어당 약 19 req/s)

7건 = 이전 상태 조회 1 + 체크포인트 3개 테이블 저장 3 + 요청 끝 삭제 3

데이터 마이그레이션 · 개요

2025. 6 ~ 2026. 6

그룹웨어 3.0 → 4.0 데이터 마이그레이션

VM 기반 모놀리식(3.0)의 DB·첨부·메일 데이터를 k8s 기반 차세대(4.0, DB + GCS)로 옮기는 Airflow 파이프라인
공유형: 서비스 담당자 약 10명이 쿼리 작성, DBA 검토 · 단독형: 서비스 담당자 없이 3명이 전 과정 진행

	1단계 공유형 (2025.6 ~ 2026.2)	2단계 단독형 (2026.1 ~ 2026.6)
대상	공유 클러스터 9개 전체, 3차례 단계 이관 (1 → 2 → 6개) 총 약 20만 명	단독형 고객 2곳 → 4.0 단독형 진행 중 요구사항 변경(공유형 통합 → 단독 유지)
실행 구조	Airflow 1대, 단일 메인 DAG	master / worker 분산 고객 VPC의 worker가 변환, master가 병합·적재
네트워크	사내망, 중앙 DB 직접 처리	고객 VPC 안에서 변환, 중앙 DB는 SSH 터널 경유
내 담당	Airflow 파이프라인·첨부 이관 **주 개발** 서비스 담당자 쿼리 실행·검증 결재·게시판 본문 변환 Python 구현	그룹웨어 서비스 이관 (ID 채번 변경·PK 처리) 이관 정책·연동·후속작업 플로우 자동화 경리회계·첨부 이관, 운영 장애 대응

Apache Airflow · PostgreSQL · MariaDB · GCS (FUSE) · rclone · Python

데이터 마이그레이션 · 문제 해결

1단계 공유형

중단·실패해도 다시 돌릴 수 있는 첨부 이관 파이프라인

① 문제

- 클러스터당 약 20TB, 총 약 200TB 첨부를 VM 디스크 → GCS로 이관, 오래 걸린 곳은 70시간 이상
- gsutil로는 클러스터 하나(20TB)에 약 2주 예상, 본이관 일정에 맞출 수 없음
- 중간에 멈추면 어디까지 됐는지 알 수 없음

② 원인

- 업로드가 상태 없는 일회성 명령
- 업로드 성공과 메타데이터 반영 성공이 따로 놂
- 여러 VM 디스크를 합치며 GCS 경로 충돌

③ 해결

- 파일별 상태 기록 + 실패 4유형 구분 → 실패분만 재시도 DAG
- GCS FUSE + cp -n으로 재실행 안전, 500개 배치
- 업로드 후 GCS API로 존재·크기 검증
- {회사}/{날짜}/{파일} 경로 매핑, 충돌은 자동 교정

④ 결과

미이관
0.2%

- 미이관 0.2%는 모두 사용하지 않는 파일이라 이관 대상에서 제외
- 클러스터 9개 첨부 이관 완료 (1 → 2 → 6개 차수)
- 재실행해도 이미 올린 파일은 건너뛰고 실패분만 처리

선이관(본이관 1주 전 시작) → 증분 이관(운영 중 추가된 파일만) → 본이관 당일엔 **잔여분만** 처리

메일 스토어 · 개요

2024. 7 ~ 2025. 6

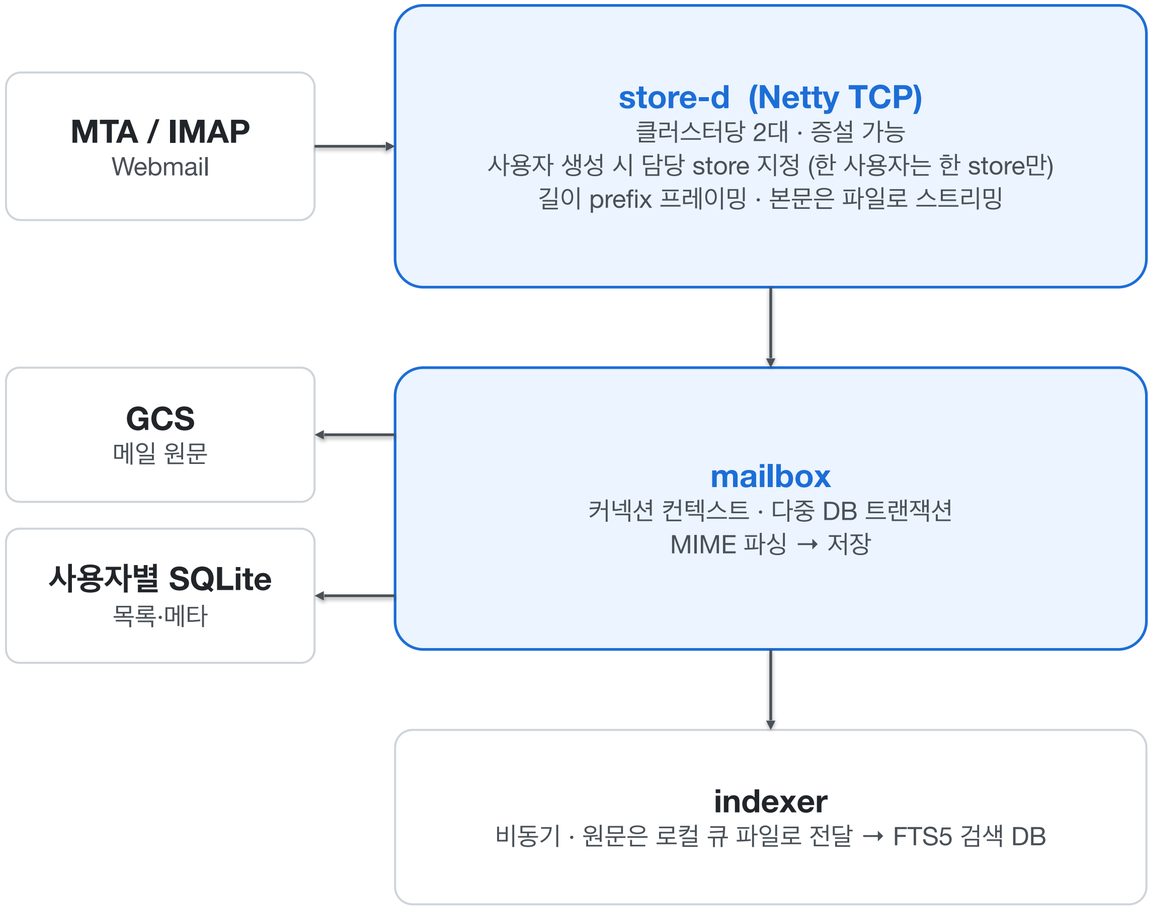
메일 스토어 신규 구축 (C → Java)

배경

- 디스크 확장 한계 → 메일 원문 저장소를 Local → GCS로
- 기존 C 기반 스토어를 Java로 재구축

역할 — 외주와 협업, Java 구조는 본인

- 외주(C 개발 경험)가 메일 API 요구사항·전체 아키텍처 설계
- **C 중심 설계를 Spring 구조에 맞게 함께 재설계** (Apache James 참고)
- **사용자별 SQLite 커넥션·트랜잭션을 Spring tx 방식(AOP + ThreadLocal)으로 직접 구현, 메일 append 수신 개발**
- 개발 후 바로 마이그레이션에 투입, 운영은 다른 담당자가 이어받음



Java · Spring · Netty · MyBatis · SQLite (FTS5) · GCS

메일 스토어 · 문제 해결

사용자별 **SQLite**를 안전하게 다루는 커넥션 구조와 **append** 수신

① 문제

- 사용자마다 개인 SQLite 4종 (메일 mindex·대용량 첨부 ·MDN·웹폴더)
- TCP 채널마다 대상 사용자가 다름
- 대용량 메일을 받을 때 메모리 부담

② 고려할 점

- DB를 매번 열고 닫는 비용
- '나에게 쓰기'처럼 한 사용자 mindex에 쓰기가 동시에 들어오면 → busy lock
- 여러 DB에 걸친 쓰기 중 실패하면 함께 되돌려야 함
- 수신 실패 시 받던 파일 정리

③ 해결

- 커넥션 컨텍스트가 DB 종류별로 필요할 때만 열고, 없으면 스키마 생성
- Netty 채널에 묶어 재사용, 다른 사용자 요청은 차단
- Spring tx 방식을 AOP + ThreadLocal로 직접 구현 → mindex·MDN을 함께 begin·rollback
- append: 헤더 → continuation → 본문을 파일로 스트리밍

④ 결과

Java 스토어 4.0 전환

- C 스토어를 대체하는 4.0 메일 스토어, 클러스터 9개 × 2대로 마이그레이션 이후 전 고객이 사용
- 쓰기는 시작부터 EXCLUSIVE 락으로 순서를 세우고, 3초 넘으면 MTA가 재전송

한계: 커밋 단계 원자성은 미보장 (mindex 커밋 후 MDN 커밋 실패 시 불일치) — 기존 C와 같은 구조, 수신 확인(MDN)에 한정돼 수용

성능 안정화 · 개요

2023. 11 ~ 2024. 6

1만 명 대형 고객사 서비스 성능 안정화

배경

- 10,000명 고객사 서비스 오픈 전 안정성 확보 (부문 KPI 과제)
- 1차 서버 개선 10건(조직도·대화방·알림 등) → 2차 개선, **영향 큰 순서로 처리하고 목표 달성 후 중단**
- 출발점: 2023년 약 3,400명 고객사 장애 때 쿼리로 개선한 조직도 API도 1만 명 부하는 감당 못 함 → **변경분 대신 전체 조직도를 캐시로** 내려주는 방향 (부록)

역할 — 부하 테스트 전 과정 단독 진행

- 테스트 장비 구축, 고객사 환경 재현 (유저 12,000 · 부서 3,000)
- JMeter 출근 시나리오 작성, Scouter로 병목 분석
- 쿼리·캐시·구조 개선 → 테스트 장비 → 포탈 → 클라우드 8개 반영
- 기능 테스트는 품질관리팀·모바일개발팀과 협업

목표

1만 명 고객사 정상 서비스

출근 피크 기준 8,000명 / 10분

9시 출근 약 6,500명 + 여유 약 20% (분당 800명)
기존 최대 피크(분당 60명)의 약 13배 · 출근 1회에 API 44개

고객 환경

등록 9,724명(테스트는 12,000명) · 530개 조직 9단계 · 부서 최대 400명
9시 출근 약 6,500명 · GPS 기반 모바일 출근
개선 전: 근태 표시 ON 시 8,000명 부하에서 장애 (CPU 85%)

2차 개선 중 직접 처리한 3건 (다음 장은 출근 피크 핵심만)

1 채팅·알림 쿼리 개선 (페이징 총 개수 제거)	2 근태 표시 성능 저하	3 /ehr/timeline/info 지연

Java · Spring · JPA · MyBatis · Ehcache (Terracotta) · JMeter · Scouter

성능 안정화 · 문제 해결

출근 피크 8,000명 / 10분 부하 대응

① 문제

- 출근 피크 8,000명 / 600초 부하에서 근태 표시 ON 시 장애 (CPU 85%)
- 근태 동기화 28초, XMPP 발송 최대 20분 지연
- 메신저·모바일 조직도 표시 40~50초
- 대화방 API 응답 1초+, 에러 발생

② 원인

- 근태: 전 사원 dirty checking + 불필요한 상세 계산
- 조직도: 엔티티 2차 캐시가 비면 60초+, 쿼리가 커넥션을 4분 넘게 점유
- 대화방: 요청당 쿼리 수백 개 (N+1)
- 공통: 장비를 키워도 그대로 → 캐시 엔트리 상한에 걸림

③ 해결

- 근태: 불필요한 근무그룹·연장근무·휴가 상세 계산 제거, 1명 출근마다 전 직원 XMPP 발송 → 1분마다 변경분만 회사별 배치 발송
- 조직도: 부서·부서원 전체 캐시 + 조직 변경 이벤트로 evict
- 대화방: N+1 API를 Native Query로 전환
- 공통: 사용자 수 구간별 캐시·힙 설정 가이드

④ 결과

8,000명 / 10분 통과

- 테스트 장비 8,000명 / 600초: 장애(CPU 85%) → 정상(CPU 71%)
- 근태 동기화 28초 → 0.6초, 조직도 40~50초 → 5초, 대화방 1,022ms → 134ms
- 실서비스(1만 명 고객사) 오픈 후 출근 피크 정상, 18시 퇴근 장애는 별도 원인으로 패치 (부록)

보유 기술

Language & Framework	Java · Spring · Python · FastAPI
Persistence & Cache	JPA · MyBatis · Ehcache (Terracotta) · Redis
Data & Pipeline	PostgreSQL · MariaDB · SQLite (FTS5) · Elasticsearch · Apache Airflow
Messaging & Network	RabbitMQ · Netty (TCP)
AI	LangGraph · LangChain · MCP · OpenAI / Anthropic / Gemini API
Infra & Observability	Kubernetes · KEDA · GCP (GCS) · Prometheus · Grafana · k6 · JMeter · Scouter

감사합니다

변재혁

APPENDIX

부록 · 기술 상세

면접에서 질문이 나오면 펼쳐 보는 자료

성능·안정성 상세

부하테스트로 병목 계층 규명

- agent만 보다가 진입점 stream을 계측 → 게이트웨이(2파드) 포화 확인
- 게이트웨이를 빼고 stream → agent 직접 연결: 응답 헤더 3.2s → 22ms, 5xx 14.1% → 0%
- 체크포인트 재설계 후 다음 병목은 stream CPU (코어당 약 19 req/s)

요청당 DB 왕복 감축

- 요청 1건의 SQL 전수 조사 → nextval은 컬럼 DEFAULT로, INSERT 후 재조회는 RETURNING으로
- 신규 대화 SQL 37 → 23 문장, 프로세스당 커넥션 8 → 5

ai-file 부하테스트 · 메모리

- OOM 원인(glibc mmap 임계값 + MuPDF store) 규명 → 메모리 989MB → 280MB, 요청당 증가 79배 감소
- PPTX 읽기 요청 1.8~3.5초 → 중앙값 약 1초 (지연의 89%였던 불필요한 재압축 제거)

부록 · AI 에이전트

ai-automation 실행 구조 재설계·구현 (진행 중)

문제

- DB 테이블을 큐로, PG NOTIFY를 신호로 사용 → 지연 재시도·TTL·DLQ를 앱 코드로 재구현, 대기량 관측 불가

MQ 중심 설계

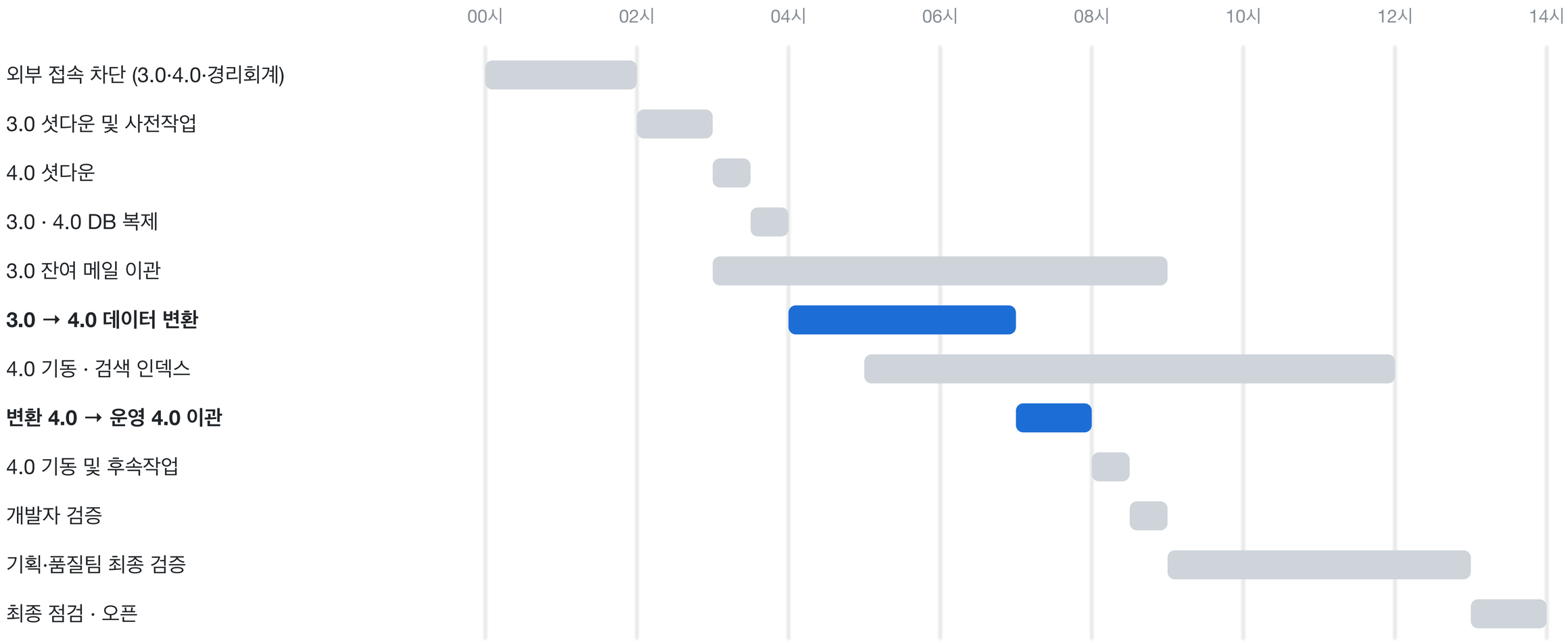
- start / continue 큐 + DLQ, DB 커밋 후 publish
- 같은 requestId 재호출에도 안전한 멱등 handoff, generation(세대 번호)로 늦게 온 메시지 차단
- Redis 전역 슬롯(소유자만 반납, epoch로 유실 감지), 회사 단위 라운드로빈으로 공정성
- 동시 실행 상한 = clamp(AI Home 파드 수 × K) → 초과분은 큐 대기
- 사용자 없는 예약 실행 인증: IAM 위임 설계 (MCP·업무 서비스 무수정)

Java tick → API → (커밋 후) start 큐 → Worker (Redis 슬롯) → AI Home StepRun → 완료 콜백 → continue 큐 → 다음 step / DLQ

부록 · 데이터 마이그레이션

1·2·3차 공통 · 00시 차단 → 14시 오픈

차수별 본이관 당일 14시간 컷오버



파란색: 내가 DAG를 순서대로 직접 실행한 구간 · 변환 실패 시 서비스 담당자에게 공유 후 재실행

기술 포인트 — 공유형

SQL 병렬 실행

- SQL을 쿼리 단위로 쪼개 쿼리 1개 = Airflow 태스크 1개 (dynamic task mapping + pool)
- 결재 본문: Python 파싱 방식으로 구현 → DBA가 이를 바탕으로 id 범위(5만 단위) SQL 방식으로 전환

첨부 실패 / 재시도

- 파일별 상태(업로드·메타데이터 성공, 실행 횟수, 결과) 기록, 실패 유형 4가지 구분
- 재시도 DAG가 실패분만 재처리, 이미 올라간 파일은 메타데이터만 재시도

gsutil → GCS FUSE

- 파티션 1개 실측으로 클러스터당 약 2주 예상 → FUSE 경로로 cp -n(재실행 안전), 500개 배치

경로 변환

- 로컬 → {회사}/{날짜}/{파일} GCS 경로 매핑, 디스크 병합 시 경로 충돌 자동 교정
- 결재 본문 HTML의 서명·이미지 경로(암호화 토큰 복호화) 치환, 원래 값 보존 + 태그 검증 DAG

부록 · 데이터 마이그레이션

기술 포인트 — 단독형

PK 처리

- 테이블군별 PK offset, JSON 설정 안에 숨은 PK(Works/Applet 멤버·부서·조건값)는 Python으로 파싱·변환·검증

성능

- chat 변환: 40만 행 × N회 상관 서브쿼리 → pre-join 1회 스캔 + 부분 인덱스
- Airflow: 오래 걸리는 태스크를 priority_weight로 먼저 실행, CPU 수 기반으로 태스크 분할 개수 계산
- 실물 파일 확인: 고객사 첨부 복제·마운트가 빠지면 이관 중단 → NFS 무응답은 별도 프로세스 + 10초 타임아웃으로 '없음'과 구분

정합성 버그

- "실패인데 성공으로 기록": sudo로 인증 환경 소실 + stderr 정규식 판정 → exit code 판정, 회귀 테스트 15건
- 운영 대응: HR 코드 길이 초과 500 오류, 대상 반전 버그, 403/401 원인을 프론트·백엔드로 입증

2025. 11 (3주)

k8s 전환 후 모바일 캘린더·연락처 동기화 장애

① 문제

- VM에서 되던 모바일 캘린더·연락처 동기화가 k8s 전환 후 미동작
- 클러스터 1~9 확인 결과 해당 기능 사용자 약 2%
- 이관 3주 전까지 담당 부서에서 미해결

② 원인

- Ingress rewrite로 클라이언트 요청 URL과 CalDAV 응답 href가 달라져 오류
- rewrite 때문에 일부 요청은 서버까지 도달하지 못함

③ 해결

- 계층별 요청 경로를 추적해 불일치 지점을 특정하고 모바일·인프라·IAM 담당자와 공유
- 본인은 CalDAV 응답 수정, IAM·모바일은 각 담당자가 수정

④ 결과

동기화 정상화

- 캘린더·연락처 동기화 정상화
- 이관 전에 사용자 약 2%의 동기화 장애 해소

부록 · 고객사 장애 대응

2023. 3 ~ 9

약 3,400명 고객사 출근 시간 장애 — 조직도 API

① 문제

- 출근 시간 조직도 API 응답 08:00 10초 → 08:20 30초 → 08:30 장애
- 3월 초·4월 초 반복 발생
- 스레드 덤프로 원인 후보 정리, Tika·ImageMagick 등은 배제

② 원인

- 조직도 변경 조회: 기준 시각이 없으면 전체 유저 조회 + 엔티티 변환 중 추가 쿼리
- 부서 조회: 큰 limit로 풀스캔, 부서원 테이블 인덱스 없음
- 근태 동기화: 전 사원 IN 쿼리, 쓰기 트랜잭션

③ 해결

- 메신저 전용 유저 컨버터, 연관관계 Lazy, 불필요 조인 제거
- 부서원 테이블 인덱스 추가, 페이징 조정 (DBA 검토)
- 근태: 전 사원 IN → company_id 조건, 조회 트랜잭션 분리

④ 결과

TPS
0.7 → 8

- nGrinder: 부서 API TPS 0.7 → 8
- 유저 API 평균 3.16초 → 1.34초 (80 vuser)
- 이 쿼리 개선판으로 1만 명 부하테스트 → 감당 못 함 → 이후 전체 조직도 캐시 방식으로 전환

부록 · 성능 안정화

2024. 6. 7 접수 → 6. 18 패치

오픈 직후 매일 **18**시에 반복된 퇴근 시간 장애

① 문제

- 오픈 직후 18시 퇴근 시간마다 장애 (6/3~6/12 중 7일)
- notifier CPU 급증 → 커넥션 풀 Max 도달, 고객 접속 불가
- 유저 11,530명(active 7,672명), 1분에 API 9,116건

② 원인

- notifier GC가 CPU를 과점유 → Web이 쓸 CPU 부족
- 모바일 접속마다 주소록(roster) 요청 → 1만 명 정보 반환
- 아이폰 요청에 JID 리소스(ios) 누락 → routing table 탐색 중 메모리 과점유
- 힙 덤프·iOS/Android 패킷 비교로 특정

③ 해결

- **notifier 인터셉터 패치 (GoCustomInterceptor)**: roster 요청 차단, presence는 PC 세션만 통과(블랙리스트 → 화이트리스트)
- 먼저 시도: presence 패킷 차단(효과 없음), CPU 12 → 36 vCPU, heap 8G → 16G
- 아이폰 리소스 누락 근본 수정은 모바일팀에 전달

④ 결과

GC 1분
180회 → 6회

- 증설(6/14) 이후 패치 전날·다음 날 비교: 18:00~18:10 openfire CPU 평균 36.6% → 7.5%, 최대 82% → 14%
- 18시 반복 장애 해소

부록 · 성능 안정화

개선 전후 측정

개선 항목	개선 전	개선 후	측정
메신저·모바일 조직도 표시	40~50초 (사용 불가)	5초 이내	테스트 장비
조직도 API 응답 (캐시)	215ms	22ms (-90%)	포탈, Scouter
조직도 API 요청당 CPU / 메모리	182ms / 78MB	9ms / 1MB	포탈, Scouter
대화방 API (Native Query)	포탈 1,022ms, 에러 88건 클라우드 평균 150ms	134ms, 에러 0건 44ms (-65%)	포탈, 클라우드 8개
대화·알림 API (Slice)	-	평균 -41%, 알림 -76%	포탈 access log
근태 표시 동기화	28초 / XMPP 최대 20분	0.6초 / 10초 이내	테스트 장비
근태 표시 ON 출근 부하 (600초, 목표 8,000명)	5,000명 정상 (CPU 56%) 8,000명 장애 (CPU 85%)	8,000명 정상 (CPU 71%)	같은 장비 64GB·8코어

부록 · 성능 안정화

기술 포인트

N+1 → Native Query

- 조직도 응답 하나에 사용자·프로필·직위/직급·사진·다국어·겸직 매핑이 필요
- fetch join은 컬렉션 2개 이상에서 MultipleBagFetchException·카테시안 곱, EntityGraph·batch size는 IN 쿼리 여러 번
- → 필요한 컬럼만 가져오는 프로젝션 쿼리 1개 + 메모리 조립, 증분 조회·Slice 페이징 (트레이드오프: DB 종속성)

조직도는 통째로, 메뉴 권한은 잘게

- 조직도: 회사 단위 트리 전체를 캐시 1건에 — 클라이언트가 트리 전체를 요청, 변경은 드물고 조회는 잦음
- 메뉴 권한: 사용자 1건 캐시 → (사용자, 메뉴) 단위로 세분화, 바뀐 메뉴 키만 evict
- 무효화: 조직 변경 이벤트 14종(기존 8종에 evict 연결 + 없던 6종 신규)으로 해당 회사 키만 evict, TTL 안전망 + 기동 시 워밍업, Terracotta eventual consistency

사이즈 가이드

- 캐시 영역별 엔트리 크기를 실측해 사용자 수 구간별(300 ~ 10,000명) 엔트리·힙 설정표 작성, 전체 고객사 반영

수신 단계 분리와 운영 이슈

수신 → 저장 → 색인 3단계

- Netty가 본문을 파일로 받고, 저장은 동기, 색인은 비동기 (indexer는 GCS 대신 로컬 큐 파일로 원문을 받음)
- 색인이 느려도 메일 수신 응답에 영향 없음, 색인 큐는 received / indexing 디렉터리로 분리 (indexing은 1024개 하위 디렉터리)

운영 이슈

- SQLite busy lock → 쓰기는 시작부터 EXCLUSIVE + 3초 timeout, 중복 수신·append 실패 잔존 파일 처리